Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ

о выполнении лабораторной работы №2
по дисциплине «Проектирование баз знаний»

на тему:

Разработка приложения для работы с реляционной базой данных

(Вариант №23)

Студент гр. 421702
А. В. Сайковский

Преподаватель
Н. А. Гуменный

Минск 2026

Содержание

Введение **3**

1 НАЗНАЧЕНИЕ И ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ **4**

2 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ **4**

3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ **5**

3.1 Слои системы 5

3.2 Прохождение HTTP-запроса 7

4 МОДЕЛЬ ДАННЫХ **7**

4.1 Основные сущности 7

4.2 Отношения между сущностями 8

4.3 Ограничения целостности 9

5 КЛАССЫ ПРОГРАММЫ **10**

5.1 Классы моделей 10

5.2 Pydantic-схемы 11

5.3 Репозитории 12

5.4 Сервисы 12

5.5 HTTP-контроллеры 13

6 ИСПОЛЬЗОВАНИЕ GENERIC-КОМПОНЕНТОВ **13**

7 ОБРАЩЕНИЕ К БАЗЕ ДАННЫХ **14**

7.1 Создание подключения и транзакции 14

7.2 Ручные SQL-запросы 15

7.3 Параметры от frontend и защита от SQL-инъекций 15

7.4 Запросы со связанными данными 16

8 ОТЧЁТНЫЕ ЗАПРОСЫ **16**

8.1 Прайс-лист поставщика на дату 16

8.2 Товары с минимальной калорийностью 17

8.3 Товары и их рецепты 17

9 DEPENDENCY INJECTION **18**

10 REST API **18**

11 РАБОТА FRONTEND **19**

12 КОНФИГУРАЦИЯ И ЗАПУСК **20**

13 ТЕСТИРОВАНИЕ **20**

14 ОБРАБОТКА ОШИБОК И БЕЗОПАСНОСТЬ **21**

15 РЕКОМЕНДУЕМЫЕ ИЛЛЮСТРАЦИИ **22**

Заключение **23**
Список использованных источников **24**

ВВЕДЕНИЕ

Цель лабораторной работы — разработать клиент-серверное приложение для ведения банка данных технологий производства пищевых продуктов. Приложение должно хранить сведения о товарах, рецептах, ингредиентах, авторах рецептов, поставщиках и накладных, предоставлять CRUD-операции и формировать аналитические отчёты.

Серверная часть реализована на Python с использованием FastAPI. Данные хранятся в PostgreSQL. SQLAlchemy применяется как асинхронный транспорт для соединений и транзакций, однако все запросы к данным написаны вручную на SQL. Входные значения, поступающие от frontend, передаются в запросы через именованные параметры. Для проверки и сериализации данных используется Pydantic, а пользовательский интерфейс написан на HTML, CSS и JavaScript.

Настоящий отчёт описывает не только результат работы программы, но и её внутреннее устройство: слои приложения, классы, отношения между сущностями, generic-компоненты, механизм доступа к базе данных, обработку HTTP-запросов и организацию тестирования.

1 НАЗНАЧЕНИЕ И ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ

Приложение предназначено для централизованного учёта технологических данных о пищевой продукции. Пользователь может просматривать, создавать, изменять и удалять следующие виды записей:

- товары и их группы;
- рецепты и авторов рецептов;
- ингредиенты и способы их подготовки;
- поставщиков;
- накладные на поставку ингредиентов.

Кроме стандартных операций предусмотрены три специализированных отчёта:

1. прайс-лист выбранного поставщика на указанную дату;
2. товары с минимальной калорийностью рецепта;
3. перечень товаров со связанными с ними рецептами.

Frontend не обращается к PostgreSQL напрямую. Он отправляет HTTP-запросы к REST API, получает JSON и строит таблицы на странице. Благодаря этому правила валидации, SQL-запросы и обработка ошибок сосредоточены на сервере.

2 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Технология	**Назначение**
Python 3.11+	Основной язык серверной части.
FastAPI	Маршрутизация HTTP-запросов, dependency injection и формирование OpenAPI-схемы.
Pydantic 2	Валидация входных JSON-данных и сериализация ответов.

Технология	Назначение
PostgreSQL 17	Реляционное хранение данных, ограничения целостности, соединения и агрегирование.
SQLAlchemy AsyncIO	Пул соединений, асинхронная сессия, транзакции и выполнение ручного SQL.
asyncpg	Асинхронный драйвер PostgreSQL.
HTML, CSS, JavaScript	Пользовательский интерфейс и вызов API функцией `fetch`.
Docker Compose	Воспроизводимый запуск PostgreSQL и подключение seed-скриптов.
pytest, pytest-asyncio	Модульные и интеграционные проверки асинхронного кода.

3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

3.1 Слои системы

Проект построен по слоистому принципу. Каждый слой решает отдельную задачу и зависит только от нижележащих компонентов. Общая схема показана на рисунке 1.

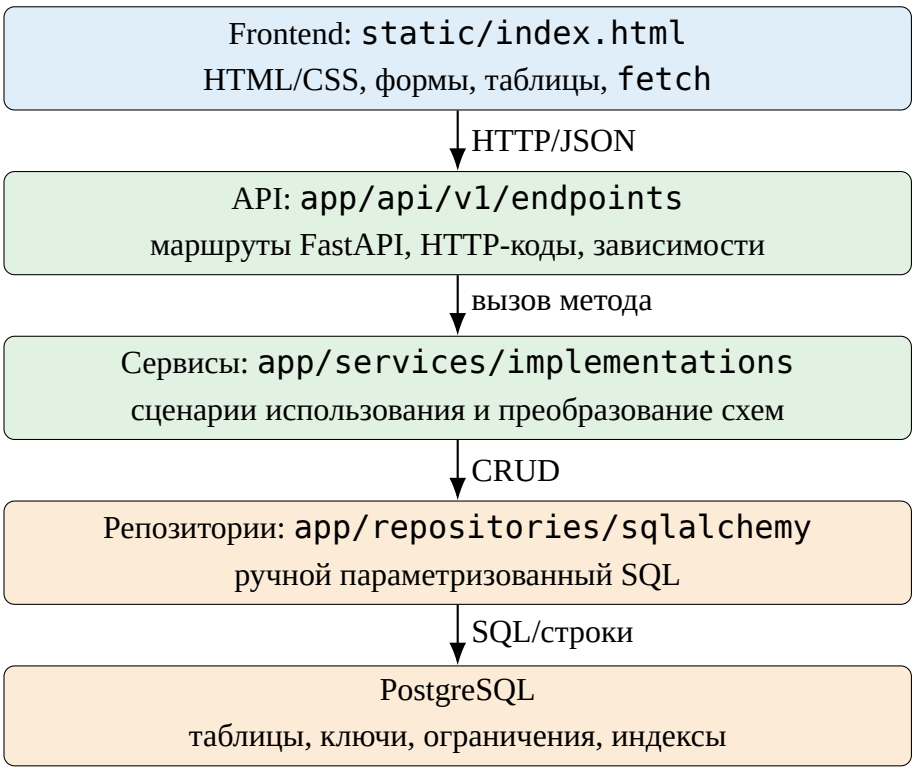


Рис. 1: Слои приложения и направление основного запроса

Назначение основных каталогов:

- `app/models` — классы структуры предметной области и метаданные таблиц;
- `app/schemas` — Pydantic-схемы запросов и ответов;
- `app/repositories` — контракты доступа к данным и SQL-реализации;
- `app/services` — прикладные операции;
- `app/api/v1/endpoints` — HTTP-контроллеры;
- `app/db` — создание сессии и фабрики зависимостей;
- `sql` — DDL-схема и начальное наполнение;
- `static` — frontend;
- `tests` — автоматические тесты.

3.2 Прохождение HTTP-запроса

Например, при открытии таблицы товаров выполняется следующая последовательность:

1. JavaScript вызывает `GET /api/v1/products/`.
2. FastAPI сопоставляет адрес с функцией `list_products`.
3. Механизм `Depends` создаёт `AsyncSession` и передаёт её в `ProductService`.
4. Сервис вызывает `ProductRepository.list`.
5. Репозиторий выполняет ручной `SELECT` с соединением таблиц `products` и `product_groups`.
6. Полученные строки преобразуются в объекты, затем в `ProductRead`.
7. FastAPI сериализует список в JSON, после чего frontend строит HTML-таблицу.

Таким образом, UI не знает структуру соединения с БД, а репозиторий не зависит от деталей HTML-представления.

4 МОДЕЛЬ ДАННЫХ

4.1 Основные сущности

В базе данных определены десять таблиц. Их назначение приведено в таблице 2.

Таблица 2: Сущности предметной области

Таблица / класс	Назначение
`product_groups` / `ProductGroup`	Справочник групп товаров.
`preparation_methods` / `PreparationMethod`	Справочник способов подготовки ингредиента.

Таблица / класс	Назначение
`ingredients` / `Ingredient`	Ингредиенты и их калорийность на один грамм.
`suppliers` / `Supplier`	Контактные данные поставщиков.
`authors` / `Author`	Авторы технологических рецептов.
`products` / `Product`	Производимые товары, относящиеся к определённой группе.
`recipes` / `Recipe`	Варианты рецептов товара с номером, описанием и автором.
`recipe_ingredients` / `RecipeIngredient`	Состав рецепта: ингредиент, способ подготовки и масса на 1 кг продукта.
`invoices` / `Invoice`	Заголовок накладной: поставщик, номер и дата.
`invoice_items` / `InvoiceItem`	Позиция накладной: ингредиент, количество и цена единицы.

4.2 Отношения между сущностями

Родитель	Связь	Дочерняя сущность	Удаление
ProductGroup	один-ко-многим	Product	RESTRICT
Product	один-ко-многим	Recipe	CASCADE
Author	один-ко-многим	Recipe	RESTRICT
Recipe	один-ко-многим	RecipeIngredient	CASCADE
Ingredient	один-ко-многим	RecipeIngredient	RESTRICT
PreparationMethod	один-ко-многим	RecipeIngredient	RESTRICT

Родитель	Связь	Дочерняя сущность	Удаление
Supplier	один-ко-многим	Invoice	RESTRICT
Invoice	один-ко-многим	InvoiceItem	CASCADE
Ingredient	один-ко-многим	InvoiceItem	RESTRICT

Таблица `recipe_ingredients` реализует отношение многие-ко-многим между рецептами и ингредиентами и одновременно хранит атрибуты связи. Аналогично, `invoice_items` связывает накладные и ингредиенты.

`CASCADE` применяется там, где дочерняя запись не имеет смысла без родителя: при удалении рецепта удаляется его состав, а при удалении накладной — её позиции. `RESTRICT` защищает справочные записи, уже используемые в бизнес-данных.

4.3 Ограничения целостности

Схема базы данных использует:

- первичные ключи `id`;
- внешние ключи для всех межтабличных отношений;
- уникальные коды справочников и номера рецептов;
- уникальность пары «поставщик–номер накладной»;
- уникальность ингредиента внутри накладной;
- ограничения `CHECK`: неотрицательная калорийность, положительные количество и цена;
- индексы по внешним ключам и дате накладной.

Часть определения позиции накладной выглядит следующим образом:

```sql
CREATE TABLE invoice_items (
    id            SERIAL PRIMARY KEY,
    invoice_id    INTEGER NOT NULL
                  REFERENCES invoices(id) ON DELETE CASCADE,
    ingredient_id INTEGER NOT NULL
                  REFERENCES ingredients(id) ON DELETE
                   ↪  RESTRICT,
    quantity      NUMERIC(14, 3) NOT NULL,
    unit_price    NUMERIC(14, 4) NOT NULL,
    CONSTRAINT uq_invoice_items_invoice_id_ingredient_id
        UNIQUE (invoice_id, ingredient_id),
    CONSTRAINT ck_invoice_items_quantity_positive
        CHECK (quantity > 0),
    CONSTRAINT ck_invoice_items_unit_price_positive
        CHECK (unit_price > 0)
);
```

5 КЛАССЫ ПРОГРАММЫ

5.1 Классы моделей

Классы из `app/models` описывают поля и связи сущностей. Все они наследуются от `Base`. Базовый класс содержит единое соглашение об именовании индексов, первичных, внешних, уникальных и проверочных ограничений.

Несмотря на декларативное описание SQLAlchemy, ORM-механизм не используется для построения запросов. Классы служат типизированным представлением данных и источником метаданных, а чтение и изменение строк выполняется ручным SQL.

Пример модели товара:

```python
class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(255),
     ↪  nullable=False)
    group_id: Mapped[int] = mapped_column(
```

```
        ForeignKey("product_groups.id", ondelete="RESTRICT"),
        index=True,
        nullable=False,
    )

    group: Mapped["ProductGroup"] = relationship(
        back_populates="products",
        lazy="selectin",
    )
```

5.2 Pydantic-схемы

Модель БД и модель API решают разные задачи. Для каждого ресурса используются отдельные схемы:

- `*Base` — общие поля;
- `*Create` — обязательные поля при создании;
- `*Update` — необязательные поля частичного обновления;
- `*Read` — ответ API с идентификатором и, при необходимости, вложенными объектами.

Например, `IngredientBase` проверяет, что калорийность неотрицательна, имеет не более 12 цифр и не более четырёх знаков после запятой. Схема `AuthorUpdate` дополнительно ограничивает год диапазоном от 1800 до 2100. Такая валидация выполняется до обращения к базе данных.

Базовая схема разрешает построение ответа из атрибутов объекта:

```
class BaseSchema(BaseModel):
    model_config = ConfigDict(
        from_attributes=True,
        populate_by_name=True,
        str_strip_whitespace=True,
    )
```

5.3 Репозитории

Репозиторий отделяет прикладную логику от конкретного SQL. Интерфейс `IRepository` задаёт пять асинхронных операций: `get`, `list`, `create`, `update` и `delete`.

`BaseRepository` реализует общий CRUD. Конкретные классы передают ему имя таблицы, модель, набор разрешённых столбцов и, при необходимости, SQL-типы параметров. Например:

```
class IngredientRepository(BaseRepository[Ingredient],
 ↪  IRepository[Ingredient]):
    def __init__(self, session: AsyncSession) -> None:
        super().__init__(
            session,
            Ingredient,
            "ingredients",
            ("name", "calories_per_gram"),
            {"calories_per_gram": Numeric(12, 4)},
        )
```

Для `ProductRepository`, `RecipeRepository` и `InvoiceRepository` методы чтения переопределены. Они используют `JOIN` и вручную собирают связанные объекты: группу товара, автора и состав рецепта, поставщика и позиции накладной.

5.4 Сервисы

Сервисы реализуют сценарии использования приложения. Они принимают Pydantic-схему, создают объект предметной области, вызывают репозиторий и возвращают схему ответа. При обновлении сервис сначала читает текущую запись, изменяет только переданные поля и затем передаёт итоговое состояние в репозиторий.

Отдельные сервисы `SupplierPriceListService`, `MinCalorieProduc` и `ProductRecipesService` реализуют отчёты, не сводимые к простому CRUD.

5.5 HTTP-контроллеры

Endpoint отвечает только за протокол HTTP: получает path-, query- или body-параметры, вызывает сервис и выбирает статус ответа. Бизнес-логика и SQL в контроллере отсутствуют.

```python
@router.get("/{product_id}", response_model=ProductRead)
async def get_product(
    product_id: int,
    service=Depends(get_product_service),
) -> ProductRead:
    obj = await service.get(product_id)
    if obj is None:
        raise HTTPException(status_code=404, detail="Product
        ↪  not found")
    return obj
```

6 ИСПОЛЬЗОВАНИЕ GENERIC-КОМПОНЕНТОВ

Generic позволяет описать алгоритм один раз и сохранить информацию о типах конкретных сущностей. В проекте объявлен параметр типа `T`, ограниченный базовым классом `Base`:

```python
T = TypeVar("T", bound=Base)

class IRepository(ABC, Generic[T]):
    @abstractmethod
    async def get(self, id: int) -> T | None:
        raise NotImplementedError

    @abstractmethod
    async def list(self, *, offset: int = 0,
                   limit: int = 100) -> list[T]:
        raise NotImplementedError
```

После подстановки `Product` выражение `IRepository[Product]` означает репозиторий, методы которого возвращают именно товары. Для `IngredientRepository` тот же общий код работает уже с `Ingredient`.

Это уменьшает дублирование и помогает IDE и статическому анализатору обнаруживать несовместимые типы.

Generic используется в трёх местах:

1. `IRepository[T]` — общий контракт репозитория;
2. `BaseRepository[T]` — единая реализация SQL CRUD;
3. `Page[T]` — универсальная схема страницы элементов.

В проекте также предусмотрены generic-контракты `ICrudService[T,C,U]` и шаблон `BaseCrudService[T,C,U,R]`. Текущие предметные сервисы реализованы явно, поскольку у разных сущностей различаются правила частичного обновления и состав схем ответа. Таким образом, общий шаблон существует как расширяемая основа, а рабочие сервисы сохраняют явную предметную логику.

7 ОБРАЩЕНИЕ К БАЗЕ ДАННЫХ

7.1 Создание подключения и транзакции

В модуле `app/db/session.py` создаётся асинхронный engine. Строка подключения формируется из `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` и `DB_PASSWORD`. Параметр `pool_pre_ping=True` проверяет соединение перед выдачей из пула.

Dependency `get_session` открывает сессию на время HTTP-запроса. При успехе транзакция фиксируется, при ошибке выполняется `rollback`.

```python
async def get_session() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_maker() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

7.2 Ручные SQL-запросы

ORM-запросы вида `select(Model)`, `session.add` или `session.get` не используются. Текст операций записывается явно. Общий запрос чтения одной строки имеет следующий вид:

```
sql = text(
    f"SELECT id, {', '.join(self._columns)} "
    f"FROM {self._table} WHERE id = :id"
)
row = (
    await self._session.execute(sql, {"id": id})
).mappings().one_or_none()
```

Имена таблицы и столбцов берутся только из кода конкретного репозитория и не могут быть заданы пользователем. Значение `id`, как и остальные данные frontend, передаётся отдельно от SQL-текста.

Создание записи выполняется оператором `INSERT ... RETURNING`. Это позволяет за один запрос получить назначенный PostgreSQL идентификатор:

```
INSERT INTO products (name, group_id)
VALUES (:name, :group_id)
RETURNING id, name, group_id;
```

Обновление и удаление также используют `RETURNING`. Если внешние ключи, уникальность или `CHECK`-ограничение нарушены, перехватывается `IntegrityError`, транзакция откатывается, а API возвращает статус 409.

7.3 Параметры от frontend и защита от SQL-инъекций

При создании товара браузер отправляет JSON, например:

```
{
    "name": "Ржаной хлеб",
    "group_id": 1
}
```

FastAPI и Pydantic проверяют типы, затем репозиторий формирует словарь параметров. Драйвер передаёт значения PostgreSQL отдельно от текста запроса. Даже если строка содержит кавычки или последовательность `DROP TABLE`, она сохраняется как обычное значение поля и не становится частью SQL-команды.

Для `Decimal` и `date` дополнительно задаются типизированные `bindparam`. Это обеспечивает одинаковое корректное преобразование в PostgreSQL и тестовой SQLite.

7.4 Запросы со связанными данными

Для отображения товара вместе с группой используется ручное соединение:

```sql
SELECT
    p.id,
    p.name,
    p.group_id,
    pg.id AS group_object_id,
    pg.code AS group_code,
    pg.name AS group_name
FROM products AS p
JOIN product_groups AS pg ON pg.id = p.group_id
ORDER BY p.id
LIMIT :limit OFFSET :offset;
```

Результат читается через `mappings()`, после чего `ProductRepository` создаёт `Product` и вложенный `ProductGroup`. Аналогично репозитории рецептов и накладных группируют несколько строк результата в один родительский объект со списком дочерних.

8 ОТЧЁТНЫЕ ЗАПРОСЫ

8.1 Прайс-лист поставщика на дату

Сервис сначала проверяет наличие поставщика, затем соединяет накладные, их позиции и ингредиенты. Условия по поставщику и дате параметризованы.

```sql
SELECT
```

```
    ing.id AS ingredient_id,
    ing.name AS ingredient_name,
    ii.unit_price,
    inv.number AS invoice_number
FROM invoices AS inv
JOIN invoice_items AS ii ON ii.invoice_id = inv.id
JOIN ingredients AS ing ON ing.id = ii.ingredient_id
WHERE inv.supplier_id = :supplier_id
  AND inv.invoice_date = :price_date
ORDER BY ing.id, inv.id;
```

Ответ включает объект поставщика, выбранную дату и список цен. Если поставщик существует, но на эту дату накладных нет, возвращается пустой список позиций.

8.2 Товары с минимальной калорийностью

Для каждого рецепта калорийность одного килограмма вычисляется как сумма произведений калорийности ингредиента на его массу:

$$K_{1\,\text{кг}} = \sum_{i=1}^{n} k_i \cdot m_i, \qquad K_{100\,\text{г}} = \frac{K_{1\,\text{кг}}}{10},$$

где k_i — калорийность одного грамма ингредиента, а m_i — количество граммов этого ингредиента на килограмм продукта.

SQL-запрос использует CTE `recipe_calories`, `SUM`, `GROUP BY` и вложенный `MIN`. Благодаря сравнению с минимальным значением возвращаются все рецепты, разделяющие первое место, а не только одна произвольная строка.

8.3 Товары и их рецепты

Третий отчёт соединяет `products`, `product_groups` и `recipes`. Строки упорядочиваются по товару и рецепту. Затем сервис создаёт словарь `products_map`, где ключом является идентификатор товара, и добавляет рецепты в соответствующий список. На выходе получается удобная для JSON и frontend иерархия «товар — рецепты».

9 DEPENDENCY INJECTION

FastAPI автоматически строит граф зависимостей. Например, фабрика сервиса товаров получает сессию из `get_session`:

```
def get_product_service(
    session: Annotated[AsyncSession, Depends(get_session)],
) -> ProductService:
    return ProductService(session)
```

Контроллер запрашивает уже готовый сервис через `Depends`. Благодаря этому endpoint не создаёт соединение вручную. Для тестов зависимость можно заменить сессией тестовой БД, не меняя код API.

Преимущества подхода:

- единый жизненный цикл сессии;
- отсутствие глобальной изменяемой сессии;
- автоматическое закрытие соединения;
- удобная подмена зависимостей при тестировании;
- слабая связанность endpoint и инфраструктуры.

10 REST API

Все маршруты имеют общий префикс `/api/v1`. Для основных ресурсов реализован единый набор операций.

Метод	Путь	Назначение
POST	`/<resource>/`	Создание записи, статус 201.
GET	`/<resource>/`	Список с параметрами `offset` и `limit`.
GET	`/<resource>/<id>`	Получение одной записи или 404.
PUT	`/<resource>/<id>`	Изменение переданных полей или 404.

Метод	Путь	Назначение
DELETE	`/<resource>/<id>`	Удаление, статус 204, либо 404.

Ресурсами являются `products`, `recipes`, `suppliers`, `authors`, `ingredients`, `product-groups`, `preparation-methods` и `invoices`.

Отчётные endpoints:

- `GET /api/v1/reports/supplier-price-list`;
- `GET /api/v1/reports/min-calorie-products`;
- `GET /api/v1/reports/products-with-recipes`.

FastAPI автоматически формирует интерактивную документацию Swagger по адресу `/docs` и альтернативную ReDoc-документацию по адресу `/redoc`.

11 РАБОТА FRONTEND

Frontend находится в одном файле `static/index.html`. Объект `tableConfigs` описывает для каждого раздела адрес API, поля формы и столбцы таблицы. Это устраняет необходимость писать отдельный JavaScript-код для каждого справочника.

При загрузке документа функция `loadAllTables` последовательно вызывает `loadTable`. Полученные массивы JSON передаются функции `renderTable`. Создание и изменение выполняются формой в модальном окне, а после успешного запроса соответствующая таблица загружается заново.

```
const response = await fetch(
    `${API_BASE}${config.apiPath}/`
);
if (!response.ok) {
    throw new Error('Failed to fetch');
}
const data = await response.json();
renderTable(tableName, data, config);
```

Корневая функция FastAPI отдаёт этот файл, а каталог `/static` подключён через `StaticFiles`. Поэтому API и интерфейс работают с одного origin и не требуют отдельного frontend-сервера.

12 КОНФИГУРАЦИЯ И ЗАПУСК

Настройки читаются из файла `.env` классом `BaseSettings`. Секретные значения не зашиты в исходный код. Пример структуры файла:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=food_products
DB_USER=postgres
DB_PASSWORD=<password>
DEBUG=false
```

Docker Compose запускает PostgreSQL, публикует порт, создаёт постоянный volume и подключает два init-скрипта. `01_schema.sql` создаёт структуру, `02_seed.sql` наполняет таблицы демонстрационными данными. Seed является идемпотентным: повторное выполнение использует `ON CONFLICT` и `NOT EXISTS`, поэтому не создаёт дубликаты.

Типичная последовательность запуска:

```
docker compose up -d
python -m uvicorn app.main:app --reload
```

После этого интерфейс доступен по адресу `http://localhost:8000/`, а Swagger — по адресу `http://localhost:8000/docs`.

13 ТЕСТИРОВАНИЕ

Тесты используют SQLite в памяти, что позволяет быстро проверять SQL-логику без отдельного тестового сервера. Метаданные `Base.metadata` создают таблицы перед тестами, а fixture предоставляет `AsyncSession`.

Проверяются следующие сценарии:

- создание товара через сервис;

- чтение, обновление, список и удаление через общий SQL-репозиторий;
- безопасное сохранение строки, внешне похожей на SQL-инъекцию;
- запись значений `Decimal`;
- загрузка автора, состава рецепта, поставщика и позиций накладной;
- выполнение трёх отчётных запросов;
- базовая доступность маршрута API.

Дополнительно скрипт `scripts/smoke_postgres.py` выполняет проверку на реальной PostgreSQL: читает все коллекции, создаёт товар, читает его вместе с группой, изменяет, удаляет и вызывает три отчёта. Тестовая запись после проверки удаляется.

14 ОБРАБОТКА ОШИБОК И БЕЗОПАСНОСТЬ

Защита приложения обеспечивается на нескольких уровнях:

1. Pydantic отклоняет неверные типы, слишком длинные строки и недопустимые числовые значения со статусом 422.
2. Именованные SQL-параметры исключают интерпретацию пользовательских строк как SQL-кода.
3. Внешние ключи, уникальные ограничения и `CHECK` защищают данные независимо от API.
4. Ошибка целостности преобразуется в HTTP 409, а отсутствующая запись — в HTTP 404.
5. При исключении выполняется откат транзакции.
6. Пароль БД хранится в `.env`, который не должен включаться в систему контроля версий.

Разрешённые имена таблиц и столбцов задаются в конструкторах репозиториев. Они не принимаются из URL или JSON. Это важно, поскольку bind-параметры могут защищать значения, но не SQL-идентификаторы.

15 РЕКОМЕНДУЕМЫЕ ИЛЛЮСТРАЦИИ

В отчёт при необходимости можно добавить следующие скриншоты:

1. главная страница с заполненными таблицами;
2. форма создания или изменения товара;
3. таблица рецептов;
4. Swagger UI со списком endpoints;
5. успешный вызов одного CRUD endpoint;
6. результат каждого из трёх отчётов;
7. таблицы PostgreSQL в pgAdmin;
8. результат запуска `pytest` и PostgreSQL smoke-теста.

Скриншоты не являются частью исходного `main.tex`; их можно добавить позже стандартным окружением `figure` и командой `\includegraphics`.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработано асинхронное клиент-серверное приложение для банка данных технологий производства продуктов. Реализованы реляционная схема PostgreSQL, REST API, браузерный интерфейс, CRUD-операции и три аналитических отчёта.

Программа разделена на модели, схемы, репозитории, сервисы и HTTP-контроллеры. Generic-компоненты позволили повторно использовать типизированный CRUD-код. Обращения к базе данных выполняются ручными параметризованными SQL-запросами, а SQLAlchemy отвечает только за асинхронные соединения, выполнение команд и транзакции. Целостность поддерживается одновременно средствами Pydantic и ограничениями PostgreSQL.

Автоматические тесты проверяют стандартные операции, связанные данные, защиту параметров и отчёты. Дополнительный smoke-тест подтверждает работу полного HTTP-цикла с реальной PostgreSQL. Полученная архитектура позволяет расширять систему новыми ресурсами и отчётами без смешивания интерфейса, бизнес-логики и доступа к данным.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] PostgreSQL 17 Documentation. — https://www.postgresql.org/docs/17/.

[2] FastAPI Documentation. — https://fastapi.tiangolo.com/.

[3] SQLAlchemy 2.0 Documentation. — https://docs.sqlalchemy.org/en/20/.

[4] Pydantic Documentation. — https://docs.pydantic.dev/latest/.

[5] Docker Compose Documentation. — https://docs.docker.com/compose/.

[6] Pytest Documentation. — https://docs.pytest.org/.